CU7 Performance Assessment — LangganKu (starter)

Fill in your name and the last four digits of your IC, then work through the Tasks of the paper **in this notebook**. First create your own data file in a terminal: `python make_my_data.py <L4>` (the file `subscriber_churn.csv` appears in this folder). Keep the notebook runnable from top to bottom.

In [1]:
```
import warnings; warnings.filterwarnings("ignore")
import mlflow, numpy as np, pandas as pd

NAME = "JOASH KUMARARAJ KUMARATHEVAN"      # e.g. "AISYAH BINTI AHMAD"
L4   = "0425"                # last four digits of your IC, keep the leading zero

# ---- your personal values (rules in the paper) ----
SEED      = int(L4)
TEST_SIZE = 0.25
DEPTH     = 3
C1        = 0.12
C2        = 1.2
RANK      = 'test_recall'
PORT      = 5125
EXP       = 'LangganKu_Churn_0425'
REG       = 'sqlite:///langganku_0425.db'
```

Task 1 — Set-up, split and pipeline

In [2]:
```
mlflow.set_tracking_uri(REG)
mlflow.set_experiment(EXP)
```

```
2026/10/20 12:17:14 INFO mlflow.store.db.utils: Creating initial MLflow database tab
les...
2026/10/20 12:17:14 INFO mlflow.store.db.utils: Updating database tables
2026/10/20 12:17:19 INFO mlflow.tracking.fluent: Experiment with name 'LangganKu_Chu
rn_0425' does not exist. Creating a new experiment.
```

Out[2]:
```
<Experiment: artifact_location='file:C:/Users/User/OneDrive/Desktop/Me/Forward-Col
lege/Sem-3/CU7/PA/STUDENT_PACK/mlruns/1', creation_time=1792469839041, effective_t
race_archival_retention=None, experiment_id='1', last_update_time=1792469839041, l
ifecycle_stage='active', name='LangganKu_Churn_0425', tags={}, trace_location=Non
e, workspace='default'>
```

In [3]:
```
df = pd.read_csv('subscriber_churn.csv')
```

In [4]:
```
df.shape
```

Out[4]:
```
(1200, 10)
```

In [5]:
```
df.dtypes
```

Out[5]:
```
subscriber_id      object
payment_method     object
main_device        object
monthly_fee_rm    float64
avg_hours_week    float64
plan               object
support_calls       int64
tenure_months       int64
contract           object
churned             int64
dtype: object
```

In [6]:
```
df.isna().sum()
```

Out[6]:
```
subscriber_id     0
payment_method    0
main_device       0
monthly_fee_rm    0
avg_hours_week    0
plan              0
support_calls     0
tenure_months     0
contract          0
churned           0
dtype: int64
```

In [7]:
```
df["churned"].value_counts()
```

Out[7]:
```
churned
0    949
1    251
Name: count, dtype: int64
```

In [8]:
```
df["churned"].value_counts(normalize=True)
```

Out[8]:
```
churned
0    0.790833
1    0.209167
Name: proportion, dtype: float64
```

In [9]:
```
X = df.drop(columns=["subscriber_id", "churned"])
y = df["churned"]
```

In [10]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=TEST_SIZE, rand
```

In [11]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()
```

```
def make_pipeline(model):
    pre = ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
        ("num", "passthrough", num_cols),
    ])
    return Pipeline([("prep", pre), ("model", model)])
```

In [12]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.linear_model import LogisticRegression

def get_metrics(model, X, y):
    pred = model.predict(X)
    return {
        "accuracy":  accuracy_score(y, pred),
        "precision": precision_score(y, pred, zero_division=0),
        "recall":    recall_score(y, pred, zero_division=0),
        "f1":        f1_score(y, pred, zero_division=0),
    }

pipe = make_pipeline(LogisticRegression(max_iter=1000))
pipe.fit(X_train, y_train)

get_metrics(pipe, X_test, y_test)
```

Out[12]:
```
{'accuracy': 0.8,
 'precision': 0.5116279069767442,
 'recall': 0.36065573770491804,
 'f1': 0.4230769230769231}
```

Task 2 — Track the candidates in ONE experiment

In [13]:
```
def log_candidate(run_name, model, model_type, params):
    pipe = make_pipeline(model)
    with mlflow.start_run(run_name=f"{run_name}-{L4}"):
        pipe.fit(X_train, y_train)

        mlflow.log_param("model", type(model).__name__)
        mlflow.log_param("test_size", TEST_SIZE)
        mlflow.log_param("split_seed", SEED)
        mlflow.log_params(params)

        mlflow.set_tag("model_type", model_type)

        for prefix, X_, y_ in [("train", X_train, y_train), ("test", X_test, y_test
            for k, v in get_metrics(pipe, X_, y_).items():
                mlflow.log_metric(f"{prefix}_{k}", v)

        mlflow.sklearn.log_model(
            pipe,
            name="model",
            input_example=X_train.head(5),
            serialization_format="cloudpickle",
        )
```

In [14]:
```
from sklearn.ensemble import RandomForestClassifier

log_candidate("logreg-base", LogisticRegression(max_iter=1000),
              "linear", {"C": 1.0, "max_iter": 1000})

log_candidate(f"logreg-c{C1:g}", LogisticRegression(C=C1, max_iter=1000),
              "linear", {"C": C1, "max_iter": 1000})

log_candidate(f"logreg-c{C2:g}", LogisticRegression(C=C2, max_iter=1000),
              "linear", {"C": C2, "max_iter": 1000})

log_candidate(f"rf-d{DEPTH}", RandomForestClassifier(n_estimators=100, max_depth=DE
              "tree_ensemble", {"n_estimators": 100, "max_depth": DEPTH})
```

```
2026/10/20 12:17:23 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
2026/10/20 12:17:48 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
2026/10/20 12:18:03 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
2026/10/20 12:18:17 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle
or cloudpickle format requires exercising caution because these formats rely on Pyth
on's object serialization mechanism, which can execute arbitrary code during deseria
lization. The recommended safe alternative is the 'skops' format. For more informati
on, see: https://scikit-learn.org/stable/model_persistence.html
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
```

Task 3 — Choose the best run from code and justify it

In [15]:
```
runs = mlflow.search_runs(
    experiment_names=[EXP],
    order_by=[f"metrics.{RANK} DESC"],
)

table = runs[["tags.mlflow.runName", "metrics.train_recall",
              "metrics.test_recall", "metrics.test_accuracy"]]
table.columns = ["run_name", "train_recall", "test_recall", "test_accuracy"]
print(table.round(4))

best_run_name = table.iloc[1]["run_name"]
```

```
best_run_id   = runs.iloc[1]["run_id"]
print("Best run:", best_run_name)
```

```
           run_name  train_recall  test_recall  test_accuracy
0   logreg-c1.2-0425        0.5368       0.3607         0.8000
1   logreg-base-0425        0.5368       0.3607         0.8000
2  logreg-c0.12-0425        0.4474       0.3115         0.8033
3         rf-d3-0425        0.1368       0.0328         0.8000
Best run: logreg-base-0425
```

Why I chose this run (2–3 sentences):

- I chose logreg-base because it has the highest test_recal1 (0.3607), and I ranked by recall instead of accuracy because only about a fifth of subscribers churn, so a model can score high accuracy while missing most churned subscribers. It is also the exact same with logreg-c1.2, but I've just decided to go with the base version because it is simpler (tie rule). For both logistic regressions the train_recall and test_recall are gapped (0.53 vs 0.36), so they slightly overfit. The logreg-c0.12 scored 0.44 on train and 0.31 on test, while the random forest is much worse on both (0.13 train, 0.03 test), which means depth-3 trees underfit and miss most churned subscribers despite its 0.80 accuracy.

Task 4 — Register, alias and roll back

In [16]:
```
from mlflow.tracking import MlflowClient
client = MlflowClient()

MODEL_NAME = f"langganku_churn_{L4}"

base_run_id = runs.loc[runs["tags.mlflow.runName"] == f"logreg-base-{L4}", "run_id"

v1 = mlflow.register_model(f"runs:/{base_run_id}/model", MODEL_NAME)
v2 = mlflow.register_model(f"runs:/{best_run_id}/model", MODEL_NAME)
print(v1.version, v2.version)
```

```
Successfully registered model 'langganku_churn_0425'.
2026/10/20 12:18:32 WARNING mlflow.tracking._model_registry.fluent: Run with id e0ab
658c4fb1469580c5cdff89fc30cf has no artifacts at artifact path 'model', registering
model based on models:/m-88ee1b17231347d38b2eeb540a071e33 instead
Created version '1' of model 'langganku_churn_0425'.
Registered model 'langganku_churn_0425' already exists. Creating a new version of th
is model...
2026/10/20 12:18:32 WARNING mlflow.tracking._model_registry.fluent: Run with id e0ab
658c4fb1469580c5cdff89fc30cf has no artifacts at artifact path 'model', registering
model based on models:/m-88ee1b17231347d38b2eeb540a071e33 instead
1 2
Created version '2' of model 'langganku_churn_0425'.
```

In [17]:
```
client.set_registered_model_alias(MODEL_NAME, "champion", 1)
print("champion ->", client.get_model_version_by_alias(MODEL_NAME, "champion").vers

client.set_registered_model_alias(MODEL_NAME, "champion", 2)
print("champion ->", client.get_model_version_by_alias(MODEL_NAME, "champion").vers
```

```
client.set_model_version_tag(MODEL_NAME, "2", "approved_by", NAME)
client.update_model_version(MODEL_NAME, "2",
    description="Logistic regression with base params; best test_recall and simples
```

```
champion -> 1
champion -> 2
```

Out[17]:
```
<ModelVersion: aliases=['champion'], creation_timestamp=1792469912497, current_sta
ge='None', deployment_job_state=None, description=('Logistic regression with base
params; best test_recall and simplest of the 4 '
 'candidates.'), last_updated_timestamp=1792469912719, metrics=None, model_id=Non
e, name='langganku_churn_0425', params=None, run_id='e0ab658c4fb1469580c5cdff89fc3
0cf', run_link=None, source='models:/m-88ee1b17231347d38b2eeb540a071e33', status
='READY', status_message=None, tags={'approved_by': 'JOASH KUMARARAJ KUMARATHEVA
N'}, user_id=None, version=2, workspace='default'>
```

In [18]:
```
client.set_registered_model_alias(MODEL_NAME, "champion", 1)
print("after roll-back:", client.get_model_version_by_alias(MODEL_NAME, "champion")

client.set_registered_model_alias(MODEL_NAME, "champion", 2)
print("after roll-forward:", client.get_model_version_by_alias(MODEL_NAME, "champio
```

```
after roll-back: 1
after roll-forward: 2
```

In [19]:
```
chal_run_id = runs.iloc[0]["run_id"]
v3 = mlflow.register_model(f"runs:/{chal_run_id}/model", MODEL_NAME)
client.set_registered_model_alias(MODEL_NAME, "challenger", v3.version)
client.update_model_version(MODEL_NAME, v3.version,
    description="Logistic regression with C=1.2; Tied with champion logreg-base.")
```

```
Registered model 'langganku_churn_0425' already exists. Creating a new version of th
is model...
2026/10/20 12:18:32 WARNING mlflow.tracking._model_registry.fluent: Run with id 73c2
86aa44c449ae8f69417a31ea5f34 has no artifacts at artifact path 'model', registering
model based on models:/m-db140bd87c794e47aaba6fb07cd48298 instead
Created version '3' of model 'langganku_churn_0425'.
```

Out[19]:
```
<ModelVersion: aliases=['challenger'], creation_timestamp=1792469912983, current_s
tage='None', deployment_job_state=None, description='Logistic regression with C=1.
2; Tied with champion logreg-base.', last_updated_timestamp=1792469913079, metrics
=None, model_id=None, name='langganku_churn_0425', params=None, run_id='73c286aa44
c449ae8f69417a31ea5f34', run_link=None, source='models:/m-db140bd87c794e47aaba6fb0
7cd48298', status='READY', status_message=None, tags={}, user_id=None, version=3,
workspace='default'>
```

Task 5 — Serve the champion and call it

In [20]:
```
my_row = df.iloc[[int(L4) % len(df)]]
my_row.drop(columns=['subscriber_id', 'churned'], inplace=True)
champion = mlflow.sklearn.load_model(f"models:/{MODEL_NAME}@champion")

my_row_proba = champion.predict_proba(my_row)
print(my_row_proba)
```

```
client.set_model_version_tag(MODEL_NAME, v2.version, "my_row_proba", my_row_proba)
```

```
[[0.99162628 0.00837372]]
```

In [23]:
```
import requests, json

URL = f"http://127.0.0.1:{PORT}/invocations"

def predict_via_api(df):
    payload = {"dataframe_split": json.loads(df.to_json(orient="split", index=False
    r = requests.post(URL, json=payload, timeout=60)
    preds = r.json().get("predictions") if r.status_code == 200 else None
    return r.status_code, preds

status, preds = predict_via_api(my_row)
print(status, preds)
```

```
200 [0]
```

In [24]:
```
bad = my_row.drop(columns=["support_calls"])
payload = {"dataframe_split": json.loads(bad.to_json(orient="split", index=False))}
r = requests.post(URL, json=payload, timeout=60)
print(r.status_code)
print(r.text[:300])
```

```
400
{"error_code": "INVALID_PARAMETER_VALUE", "message": "Failed to predict data '  paym
ent_method main_device  monthly_fee_rm  ...      plan tenure_months  contract\n0
Card          TV           41.9  ...  Standard            25   6-month\n\n[1 rows x
7 columns]'. \nError: Failed to enforce
```